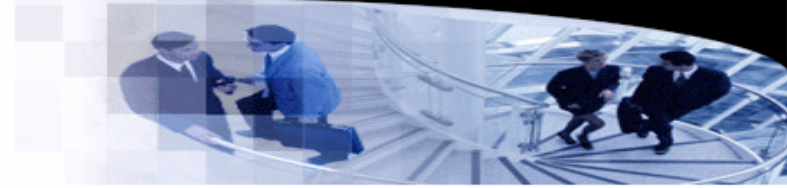




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

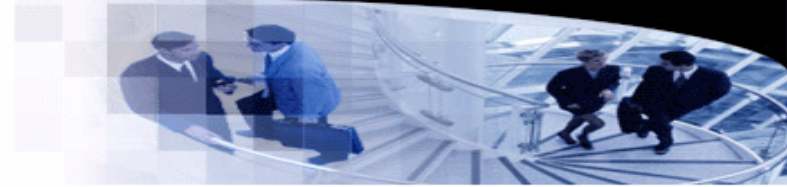
Lecture 22. Introduction of Program Analysis

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Program Analysis

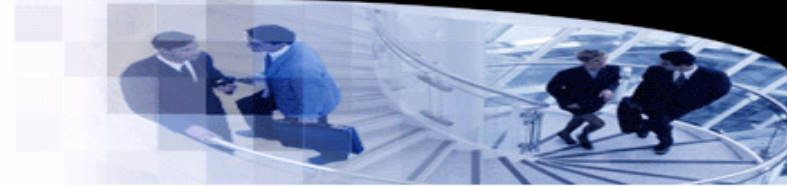


- Consider a program with a risk of division by zero
 - The compiler only reports a warning.

```
int foo(int x) {
    int y;
    switch (x) {
        case 1:
            y = x / 0;
            break;
        case 2:
            y = x / (x - 2);
            break;
    }
    return y;
}

ymt@DESKTOP-VPDNSDJ:~/analysis$ clang-14 -c div0.cpp
div0.cpp:6:19: warning: division by zero is undefined [-Wdivision-by-zero]
    y = x / 0;
              ^~
1 warning generated.
```

Program Analysis

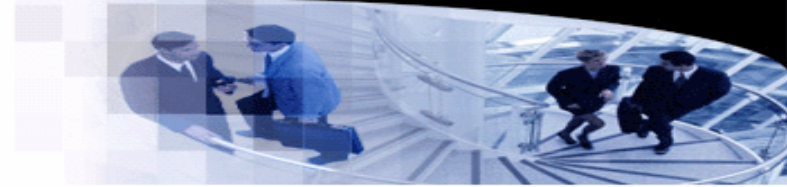


- Consider a program with a risk of division by zero
 - However, the Clang Static Analyzer can find deep errors.

```
div0.cpp:6:19: warning: Division by zero [core.DivideZero]
    y = x / 0;
           ^
div0.cpp:9:19: warning: Division by zero [core.DivideZero]
    y = x / (x - 2);
           ^
div0.cpp:12:5: warning: Undefined or garbage value returned to caller [core.uninitialized.UndefReturn]
    return y;
    ^
3 warnings generated.
```

- It is capable of performing **context-sensitive, inter-procedural** analysis.
- The list of checkers can be found on <https://clang.llvm.org/docs/analyzer/checkers.html>.

Program Analysis



- Consider a program with a risk of division by zero
- Let us try another example.

```
ymt@DESKTOP-VPDNSDJ:~/analysis$ cat div0-1.cpp

int a;

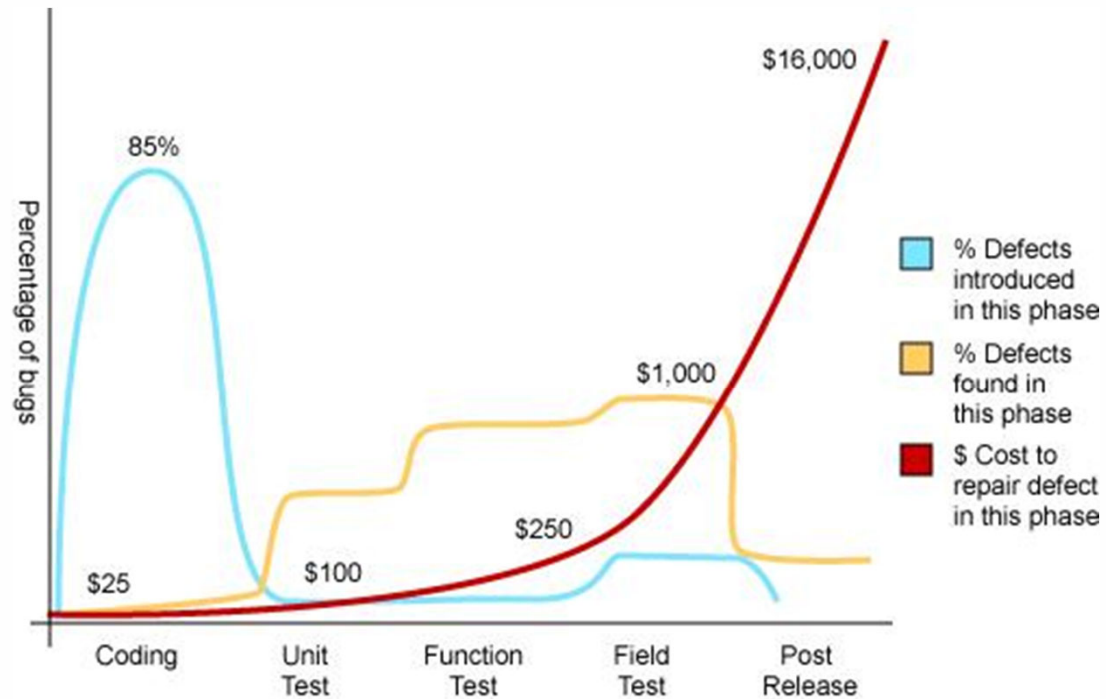
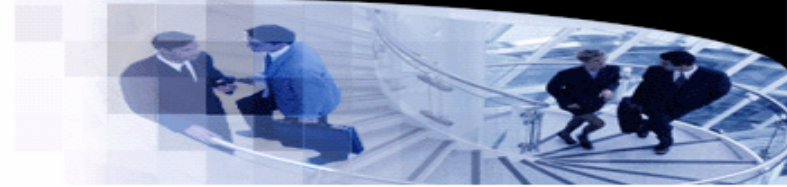
int foo(int x) {
    return x / a;
}

int bar() {
    a = 0;
    return foo(4);
}ymt@DESKTOP-VPDNSDJ:~/analysis$ clang-14 -c div0-1.cpp
ymt@DESKTOP-VPDNSDJ:~/analysis$ clang-14 --analyze div0-1.cpp
div0-1.cpp:5:14: warning: Division by zero [core.DivideZero]
    return x / a;
               ~~~~
1 warning generated.
```

- The analyzer finds an error missed by the compiler, again.

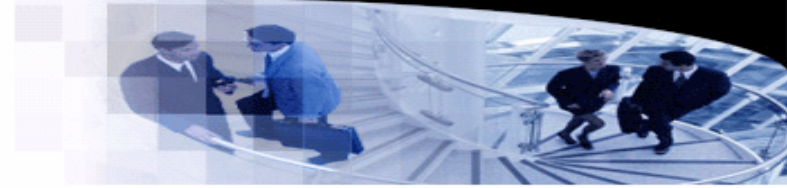
Program Analysis

- Why program analysis?
 - To find bugs as early as possible.



Source: Applied Software Measurement, Capers Jones, 1996

Program Analysis



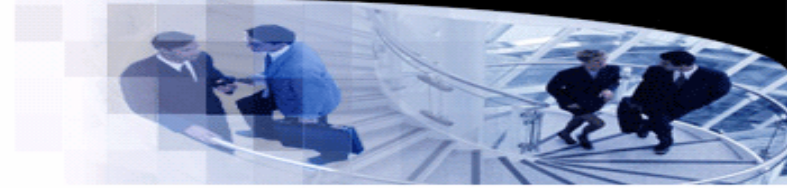
- About program analysis

- Program analysis is a systematic approach to reason about the properties of a program.
- Program analysis is usually a **static** approach, which means it does not run the program.

- Applications

- Program optimization
 - Eliminate redundant/unnecessary computations
- Verification
 - Find errors/bugs in a program
- Security
 - Privacy leak, vulnerability detection
- Program comprehension and transformation
 - Call graphs in IDE, refactoring and repair

Program Analysis



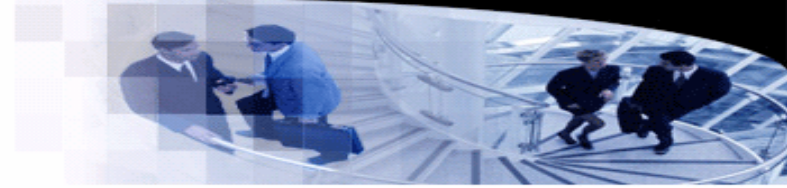
- About program analysis

- Program analysis is a systematic approach to reason about the properties of a program.
- Program analysis usually refers to **static** analysis, which means it does not run the program.

- Applications

- Program optimization
 - Eliminate redundant/unnecessary computations
- Verification
 - Find errors/bugs in a program
- Security
 - Privacy leak, vulnerability detection
- Program comprehension and transformation
 - Call graphs in IDE, refactoring and repair

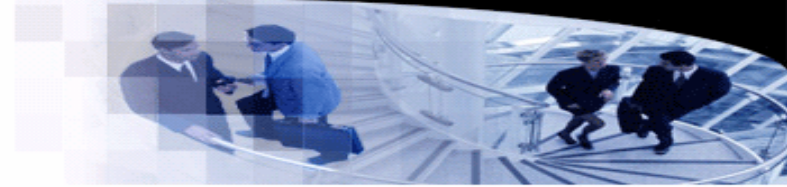
Program Analysis



- Why is static analysis challenging?

- Undecidability problem
- **Theorem:** There is no such a program M that decides for all program N , whether N terminates.
- **Proof:**
 - Suppose there is an M , which takes as input a program N and returns true if N terminates, false otherwise.
 - Now construct a new program S : if $M(S)$ then infiniteLoop().
 - If M terminates, S does not terminate.
 - If M does not terminate, S terminates.
 - Contradiction.
- Rice's theorem:
 - Any nontrivial property about the language recognized by a Turing machine is undecidable.

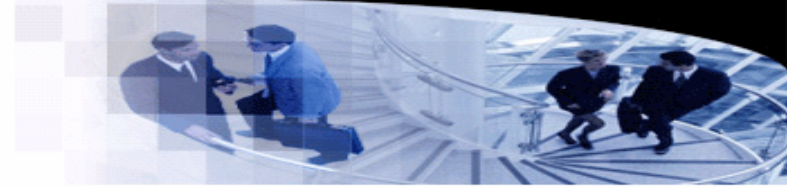
Program Analysis



- Soundness and completeness

- Soundness: Every claim made by the analyzer is true.
- Completeness: Every true claim is made by the analyzer.
- Soundness + completeness is expensive, and may be undecidable.
- Practical analyzers make approximations.

Common Analysis Tasks



- Data flow analysis

- Investigate how data flows on a program (a CFG specifically).

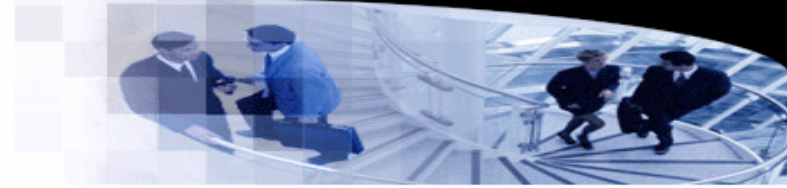
- Reaching definitions

- Claim that a **definition** of a variable v at a statement p **reaches** a statement q if there is a path from p to q such that v is not re-defined on the path.

- Applications

- Dead code elimination
 - Constant propagation
 - Copy propagation
 - Detecting uninitialized variables

Common Analysis Tasks



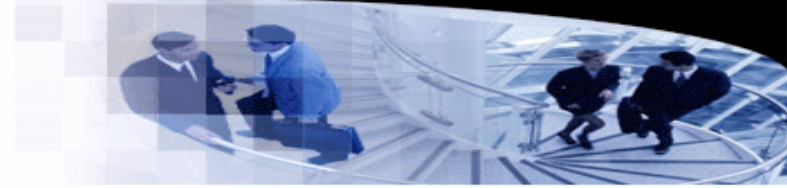
- Live variables

- Claim that a set of variables is live each program point p .

- Applications

- Register allocation
 - Garbage collection
 - Instruction scheduling
 - Dead code elimination

Common Analysis Tasks



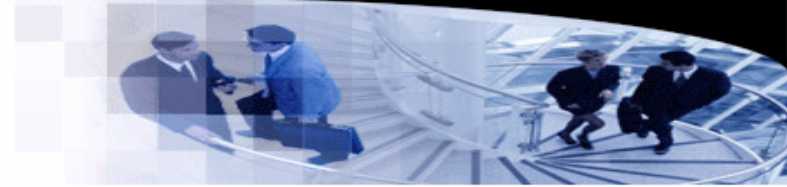
- Live variables

- Claim that a set of variables is live each program point p .

- Applications

- Register allocation
 - Garbage collection
 - Instruction scheduling
 - Dead code elimination

Common Analysis Tasks



- **Pointer analysis**
 - Claim that which pointer can point to which variable or storage location.
 - Fully precise pointer analysis is undecidable.
- **Trade-off**
 - Context sensitivity and flow sensitivity
- **Applications**
 - Data race detection
 - Alias analysis
 - Escape analysis
 - Precise control flow graphs for binaries
 - Control-flow integrity
 - Taint analysis